

plot the last *x*- and *y*-values in red with a size of 100 as well. Make sure you insert this code just before the call to `plt.show()` so the starting and ending points are drawn on top of all the other points.

When you run this code, you should be able to spot exactly where each walk begins and ends. If these end points don't stand out clearly enough, adjust their color and size until they do.

Cleaning Up the Axes

Let's remove the axes in this plot so they don't distract from the path of each walk. Here's how to hide the axes:

```
rw_visual.py  --snip--
while True:
    --snip--
    ax.scatter(rw.x_values[-1], rw.y_values[-1], c='red', edgecolors='none',
               s=100)

    # Remove the axes.
    ax.get_xaxis().set_visible(False)
    ax.get_yaxis().set_visible(False)

    plt.show()
--snip--
```

To modify the axes, we use the `ax.get_xaxis()` and `ax.get_yaxis()` methods to get each axis, and then chain the `set_visible()` method to make each axis invisible. As you continue to work with visualizations, you'll frequently see this chaining of methods to customize different aspects of a visualization.

Run *rw_visual.py* now; you should see a series of plots with no axes.

Adding Plot Points

Let's increase the number of points, to give us more data to work with. To do so, we increase the value of `num_points` when we make a `RandomWalk` instance and adjust the size of each dot when drawing the plot:

```
rw_visual.py  --snip--
while True:
    # Make a random walk.
    rw = RandomWalk(50_000)
    rw.fill_walk()

    # Plot the points in the walk.
    plt.style.use('classic')
    fig, ax = plt.subplots()
    point_numbers = range(rw.num_points)
    ax.scatter(rw.x_values, rw.y_values, c=point_numbers, cmap=plt.cm.Blues,
               edgecolors='none', s=1)
--snip--
```
